ADVENSIA

ADVancEd NetworkS and communIcAtion systems

IoT Parking Assistance
Kick-off

Dr.-Ing. Jaouhar Jemai

03.02.2026

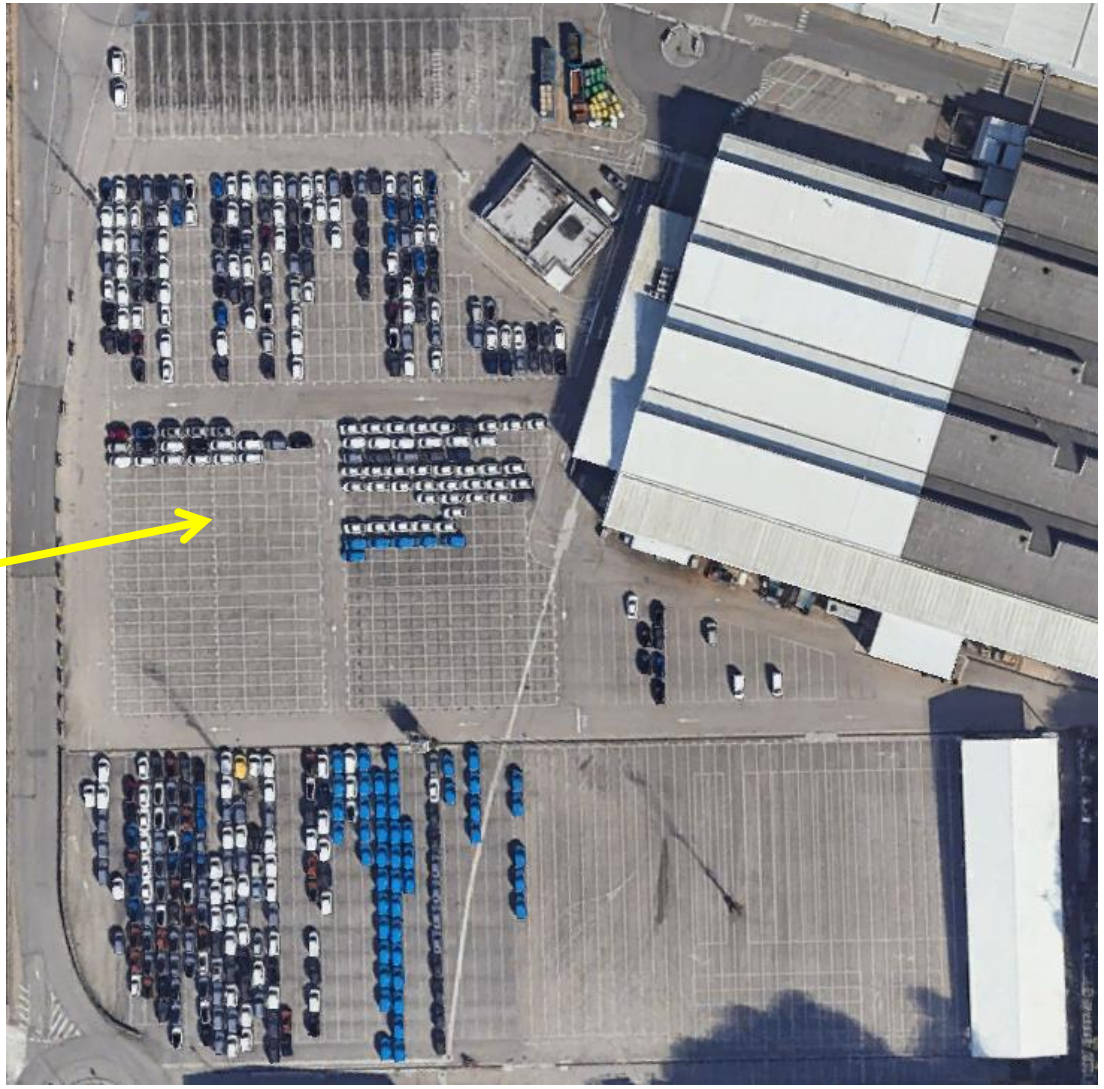
Content

- Author‘s introduction
- Customer‘s Problems to solve
- Required Front-End Functionalities
- Hardware Requirements
- Track Record
  - Our Business
  - Our Milestones
  - Track-Record over 18 Years IoT

Problems to solve

- The factory produces approximately 500,000 vehicles per year (1,400 vehicles per day) between SYS1 and SYS2.
- The vehicles leave the manufacturing plant and will be parked in the factory's parking bays.
- The parking duration ranges from 1 - 30 days within the factory and between 7 - 180 days in the port before shipping.
- The search for vehicles in the parking areas is done manually and requires resources and time.
- There is no updated overview of the locations of the vehicles and their status.

Overview



Required Front-End Functionalities

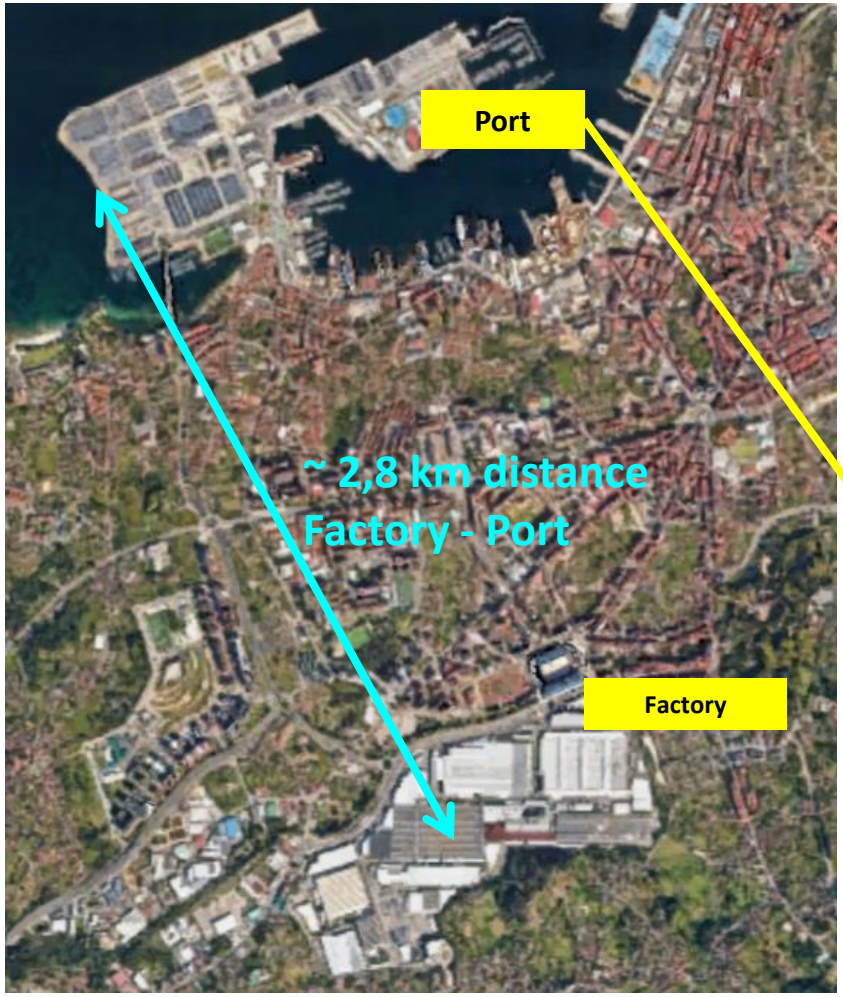
- Associate each vehicle with a tag
- Transfer vehicle attributes to the application [Type/Model, VIN, Customer, Delivery Country, Expected Delivery Date, Color]
- Display the current location of vehicles on a map
- Show a set of vehicle attributes when clicking on the vehicle object on the map [Type/Model, VIN, Customer, Delivery Country, Expected Delivery Date, Color, Parking Spot]
- Find one or more vehicles in the parking lot by searching by attribute.
- Display inventories
- Refresh should react to location changes (3s motion/2hr static)
- Trigger alerts/notifications in case of:
  - Vehicle leaves its spot unexpectedly
  - Vehicle approaches or exceeds the eligible parking period

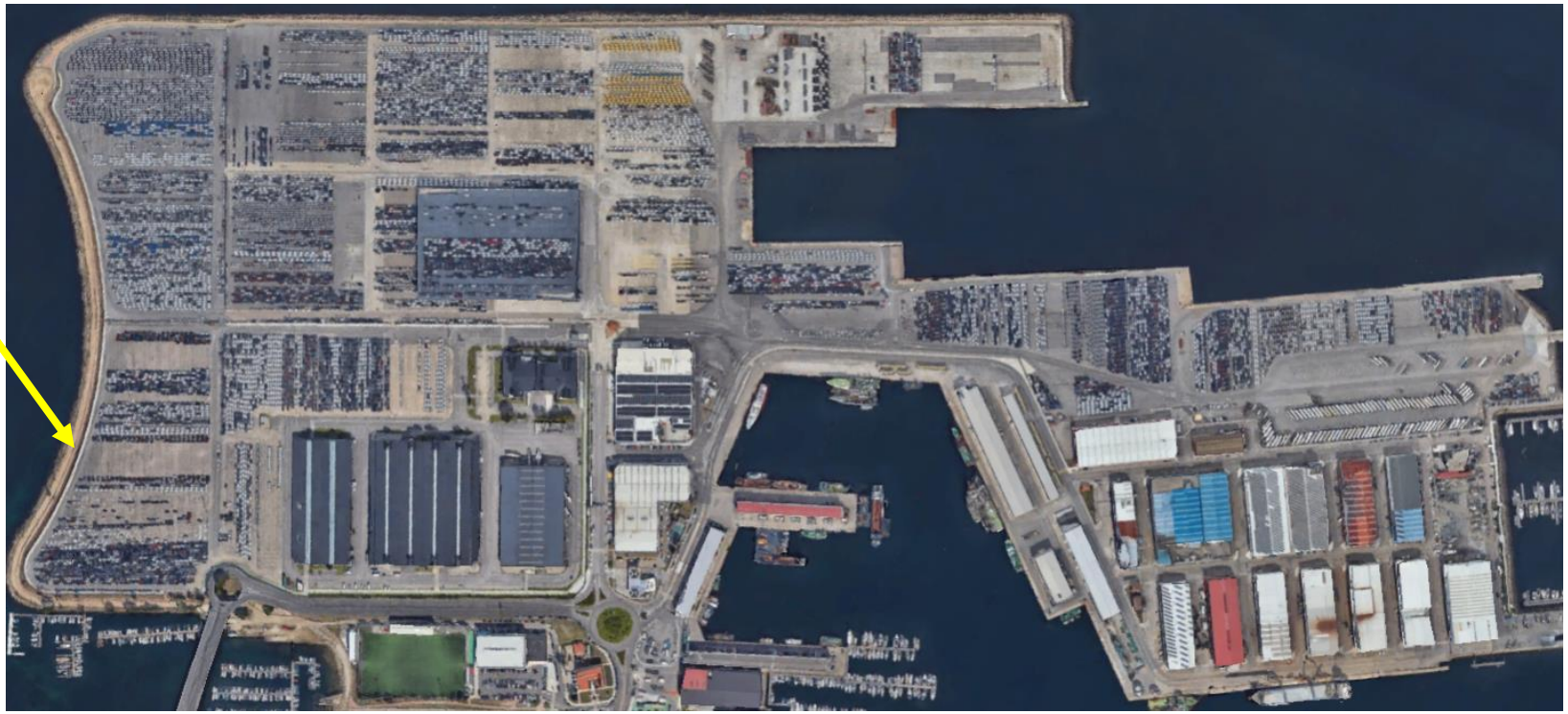
Hardware Requirements

- No external infrastructure to be installed in the parking lot.
- The tag is placed in the vehicle before leaving the building towards the outdoor parking lot (IP30 would be fine).
- The tag remains in the vehicle until it reaches the customer (e.g., vehicle dealership).
- Number of tags required: 500,000 vehicles per year → 500,000 tags per year.
- Battery lifetime > 6 months.
- Position refresh (3s motion/2hr static)
- Location accuracy ≤ 5m with 99%.

Roll-Out in the port

- Same solution to be rolled-out in the port with ~5000 cars





Non functional Requirementes

- Scalability (supporting thousands of devices, packet collision/loss)
- Self-hailing (connectivity loss, …)
- Security
- Device Maintenance (firmware OTA update)
- Network monitoring
- Hardware monitoring
  - Gateways
  - Sensors

CarPark MVP – Scope & Planning

Frontend & Backend Alignment
Target: **Demo mid-June**

MVP Objective: What we must deliver

- Demo-ready MVP by mid-June
- End-to-end flow demonstrated live:
  - End-to-end flow from association → visualization → filtering
    - Vehicle ↔ BLE Tag association
    - Vehicle list + map
    - Attribute filtering
- Focus on functionality, stability, clarity
- This is not production, not full PoC
- Create a **solid decision basis** for next phase (industrialization / rollout)

MVP Functional Scope

- Vehicle ↔ BLE Tag association
- Vehicle attributes:
  - VIN (primary key)
  - Model
  - Color
  - Fault (+ missing parts)
  - Zone
  - Delivery date
- Vehicle list + map view
- Filtering by attributes
- Vehicle counter (total / filtered)

Vehicle ↔ BLE Tag Association

- Core data model
  - One BLE Tag ↔ one Vehicle
  - Association done manually (admin / app UI)
- Mandatory fields at association
  - VIN (unique ID)
  - Model
  - Color
  - Fault status
  - Zone
  - Delivery date
  - BLE Tag ID
- Rules
  - VIN = immutable primary key
  - BLE Tag ID must be unique
  - Attributes editable after association

UI Layout

- **Single main screen**
  - **Left panel:** Vehicle list
    - VIN + key attributes
  - **Right panel:** Map
    - Vehicle markers
- **Interactions**
  - Select vehicle → highlight on map and opens attributes list
  - Select map marker → highlight in list

Filtering & Counter

- Filter
  - VIN: orig
  - Model: ori
  - Color: ori
  - Fault: manual
  - Zone: auto
  - Last seen: auto
  - Association date: auto
  - Delivery date: orig
- Behavior
  - Filters update: Vehicle list + Map + Counter
- Counter
    - Displays number of vehicles currently shown
    - Example: “42 vehicles displayed”

Frontend Scope

- Association UI
- Vehicle list component
- Map integration
- Filter panel
- Basic validation & UX
- Demo-focused (no over-polish)

Backend Scope

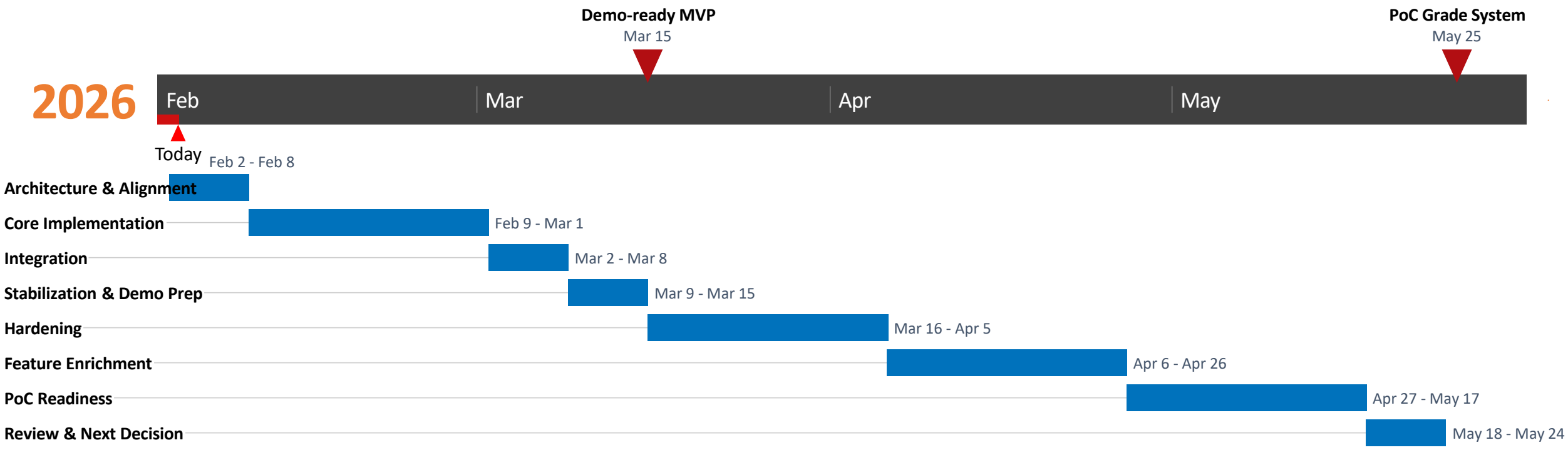
- Vehicle & BLE tag data model
- CRUD APIs
- Filtering logic
- Location ingestion & retrieval
- Stable demo backend

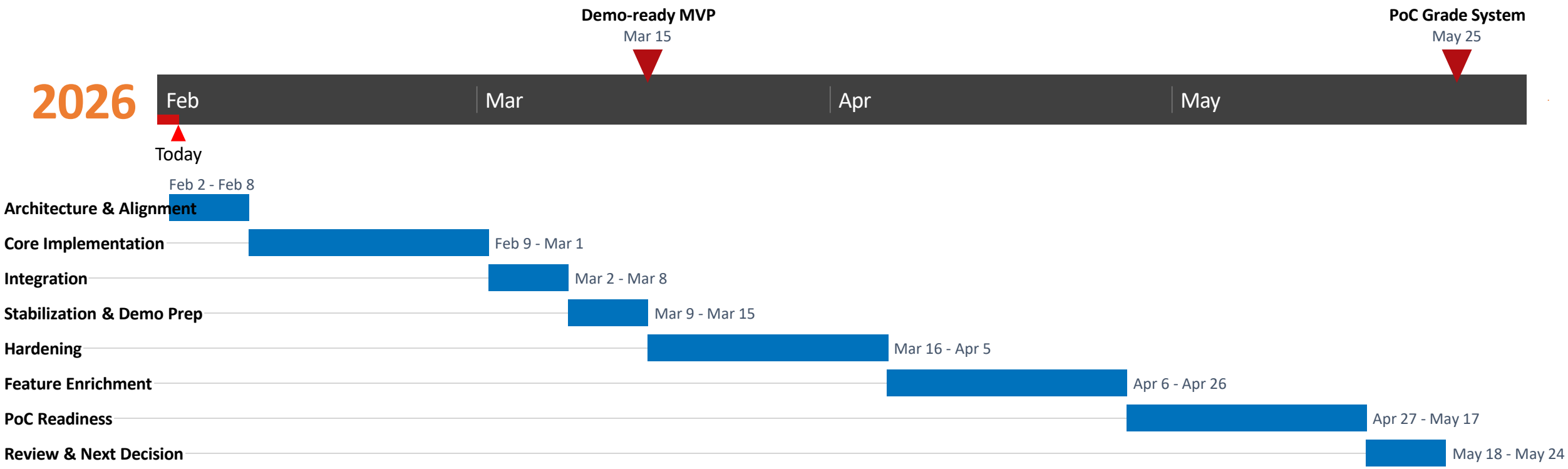
- **Explicitly not required**
  - Scaling
  - Security hardening
  - Integrations

Out of Scope

- ERP / MES integration
- User roles & permissions
- Automation workflows
- Analytics dashboards
- Production deployment

Planning


Demo-ready MVP
Mar 15
PoC Grade System
May 25
2026
Feb
Mar
Apr
May
Today
Feb 2 - Feb 8
Architecture & Alignment
Core Implementation
Feb 9 - Mar 1
Integration
Mar 2 - Mar 8
Stabilization & Demo Prep
Mar 9 - Mar 15
Hardening
Mar 16 - Apr 5
Feature Enrichment
Apr 6 - Apr 26
PoC Readiness
Apr 27 - May 17
Review & Next Decision
May 18 - May 24


Demo-ready MVP
Mar 15
PoC Grade System
May 25
2026
Feb
Mar
Apr
May
Today
Feb 2 - Feb 8
Architecture & Alignment
Core Implementation
Feb 9 - Mar 1
Integration
Mar 2 - Mar 8
Stabilization & Demo Prep
Mar 9 - Mar 15
Hardening
Mar 16 - Apr 5
Feature Enrichment
Apr 6 - Apr 26
PoC Readiness
Apr 27 - May 17
Review & Next Decision
May 18 - May 24

Planning: Phase 1: Demo-ready MVP (W1-W5)

Week	Tasks	Details
Week 1	**Architecture & Alignment**	Finalize data model (Vehicle, BLE Tag, Fault) API contracts (FE ↔ BE) Map & filtering approach Tech decisions frozen
Week 2–3	**Core Implementation**	**Backend:** ▪ CRUD APIs ▪ Filtering logic ▪ Location handling **Frontend:** ▪ Association UI ▪ Vehicle list ▪ Base map view

Planning: Phase 1: Demo-ready MVP (W1-W5)

Week	Tasks	Details
Week 4	**Integration**	▪ FE ⟷ BE integration ▪ Filtering synchronization (list + map) ▪ Vehicle counter ▪ Fault visualization
Week 5	**Stabilization & Demo Prep**	▪ Bug fixing ▪ UX cleanup ▪ Demo data set ▪ Dry-run demo

Outcome: Stable MVP demonstrating full end-to-end flow

Planning: Phase 2: Turn MVP into PoC-grade system

Week	Tasks	Details
Weeks 6–8	**Hardening**	▪ Data validation ▪ Error handling ▪ Performance cleanup ▪ Code refactoring
Weeks 9–12	**Feature Enrichment (Controlled)**	▪ Improved filtering UX ▪ Fault detail refinement ▪ Zone handling improvements ▪ Map usability improvements
Weeks 13–15	**PoC Readiness**	▪ Operational stability ▪ Data consistency ▪ Demo → pilot transition ▪ Internal documentation
Week 16	**Review & Next Decision**	▪ Technical review ▪ Lessons learned ▪ Scope decision for rollout / industrialization

Working Mode

- Weekly sync (30–45 min)
- Clear ownership FE / BE
- Short feedback loops
- Decisions documented
- MVP mindset: working > perfect

Demo Success Criteria

- Association works end-to-end
- List ↔ map synchronization works
- Filtering works reliably
- Demo runs without instability

Useful Links

- Blecon tracker
  - https://track.blecon.net/
- Blecon developer
  - https://developer.blecon.net/
- Blecon console
  - Blecon Console - Home